50 Leetcode most asked questions in Infosys

By Abhishek Rathor

Instagram: code.abhii07 (SYNTAX ERROR)

1. LC 1 – Two Sum

Description: Find two indices whose values sum to target.
Pseudocode:

```
map = empty
for i from 0 to n-1
    if target - arr[i] exists in map
        return map[target-arr[i]], i
    map[arr[i]] = i
```

2. LC 7 – Reverse Integer

Description: Reverse digits of an integer.
Pseudocode:

```
rev = 0
while x != 0
    digit = x % 10
    rev = rev * 10 + digit
    x = x / 10
return rev
```

3. LC 9 – Palindrome Number

Description: Check if number reads same backward.
Pseudocode:

```
temp = x
rev = 0
while temp > 0
    rev = rev*10 + temp%10
    temp = temp/10
if rev == x return true
else return false
```

4. LC 13 – Roman to Integer

Description: Convert Roman numeral to integer.
Pseudocode:

```
map roman values
sum = 0
for i from 0 to n-1
    if value[i] < value[i+1]
        sum -= value[i]
    else
        sum += value[i]
return sum
```

---

5. LC 14 – Longest Common Prefix

Description: Find common prefix among strings.
Pseudocode:

```
prefix = first string
for each string
    while string does not start with prefix
        remove last char from prefix
return prefix
```

---

6. LC 20 – Valid Parentheses

Description: Check balanced brackets.
Pseudocode:

```
stack = empty
for ch in string
    if opening bracket push
    else
        if stack empty or mismatch return false
        pop
return stack empty
```

---

7. LC 21 – Merge Two Sorted Lists

Description: Merge two sorted linked lists.
Pseudocode:

```
dummy node
while l1 and l2
    attach smaller node
attach remaining nodes
return dummy.next
```

---

8. LC 26 – Remove Duplicates from Sorted Array

Description: In-place remove duplicates.
Pseudocode:

```
index = 1
for i from 1 to n-1
    if arr[i] != arr[i-1]
        arr[index++] = arr[i]
return index
```

---

9. LC 27 – Remove Element

Description: Remove all occurrences of value.
Pseudocode:

```
index = 0
for i from 0 to n-1
    if arr[i] != val
        arr[index++] = arr[i]
return index
```

---

10. LC 28 – Find Index of First Occurrence

Description: Implement substring search.
Pseudocode:

```
for i from 0 to n-m
    if substring matches
        return i
return -1
```

---

11. LC 35 – Search Insert Position

Description: Find insert position in sorted array.
Pseudocode:

```
low = 0, high = n-1
while low <= high
    mid = (low+high)/2
    if arr[mid] == target return mid
    else if arr[mid] < target low = mid+1
    else high = mid-1
return low
```

---

12. LC 53 – Maximum Subarray

Description: Largest sum contiguous subarray.
Pseudocode:

```
maxSum = arr[0]
curr = arr[0]
for i from 1 to n-1
    curr = max(arr[i], curr+arr[i])
    maxSum = max(maxSum, curr)
return maxSum
```

---

13. LC 58 – Length of Last Word

Description: Length of last word in string.
Pseudocode:

```
count = 0
from end traverse until non-space
count characters until space
return count
```

---

14. LC 66 – Plus One

Description: Add one to number array.
Pseudocode:

```
carry = 1
for i from end to start
    sum = arr[i] + carry
    arr[i] = sum % 10
    carry = sum / 10
if carry insert at start
return arr
```

---

15. LC 67 – Add Binary

Description: Add two binary strings.
Pseudocode:

```
carry = 0
while i>=0 or j>=0 or carry
    sum = carry + bit1 + bit2
    append sum%2
    carry = sum/2
reverse result
```

---

16. LC 69 – Sqrt(x)

Description: Integer square root.
Pseudocode:

```
low = 1, high = x
```

```
while low <= high
    mid = (low+high)/2
    if mid*mid == x return mid
    else if mid*mid < x low = mid+1
    else high = mid-1
return high
```

---

17. LC 70 – Climbing Stairs

Description: Ways to climb stairs.
Pseudocode:

```
dp[0]=1, dp[1]=1
for i from 2 to n
    dp[i] = dp[i-1] + dp[i-2]
return dp[n]
```

---

18. LC 83 – Remove Duplicates from Sorted List

Description: Remove duplicates from linked list.
Pseudocode:

```
curr = head
while curr and curr.next
    if curr.val == curr.next.val
        skip next
    else move curr
return head
```

---

19. LC 88 – Merge Sorted Array

Description: Merge nums2 into nums1.
Pseudocode:

```
i=m-1, j=n-1, k=m+n-1
while j>=0
    place larger at k
```

---

20. LC 100 – Same Tree

Description: Check if trees identical.
Pseudocode:

```
if both null return true
if one null return false
if values differ return false
return left && right
```

---

21. LC 101 – Symmetric Tree

Description: Determine whether a binary tree is symmetric around its center.

Pseudocode:

```
checkMirror(left, right)
```

---

22. LC 104 – Maximum Depth of Binary Tree

Description: Find the maximum depth or height of a binary tree.

Pseudocode:

```
if root null return 0
return 1 + max(leftDepth, rightDepth)
```

---

23. LC 108 – Convert Sorted Array to BST

Description: Convert a sorted array into a height-balanced binary search tree.

Pseudocode:

```
mid = (l+r)/2
node = new Tree(mid)
left = build(l,mid-1)
right = build(mid+1,r)
```

---

24. LC 110 – Balanced Binary Tree

Description: Check whether a binary tree is height-balanced, meaning the height difference is not more than one.

Pseudocode:

```
if height difference >1 return false
```

---

25. LC 112 – Path Sum

Description: Check if a root-to-leaf path exists such that the sum of node values equals a given target.

Pseudocode:

```
if leaf and sum==val return true
```

```
check left or right with reduced sum
```

---

26. LC 121 – Best Time to Buy and Sell Stock

Description: Find the maximum profit by choosing a single day to buy and a later day to sell stock.

Pseudocode:

```
minPrice = INF
maxProfit = 0
for price in prices
    minPrice = min(minPrice, price)
    maxProfit = max(maxProfit, price-minPrice)
```

---

27. LC 122 – Best Time Stock II

Description: Find the maximum profit by making as many buy-sell transactions as you want.

Pseudocode:

```
profit += all positive differences
```

---

28. LC 125 – Valid Palindrome

Description: Check if a string is a palindrome considering only alphanumeric characters and ignoring case.

Pseudocode:

```
i=0, j=n-1
skip non-alphanumeric
compare lowercase chars
```

---

29. LC 136 – Single Number

Description: Given an array where every element appears twice except one, find the element that appears once.

Pseudocode:

```
result = 0
for num in arr
    result ^= num
return result
```

---

30. LC 141 – Linked List Cycle

Description: Detect whether a linked list contains a cycle.

Pseudocode:

```
slow = head, fast = head
while fast and fast.next
    slow = slow.next
    fast = fast.next.next
    if equal return true
return false
```

---

31. LC 144 – Preorder Traversal

Description: Traverse a binary tree in preorder sequence: root, left, right.

Pseudocode:

```
root, left, right
```

---

32. LC 145 – Postorder Traversal

Description: Traverse a binary tree in postorder sequence: left, right, root.

Pseudocode:

```
left, right, root
```

---

33. LC 160 – Intersection of Two Linked Lists

Description: Find the node at which two singly linked lists intersect.

Pseudocode:

```
move pointers, switch heads
```

---

34. LC 169 – Majority Element

Description: Find the element that appears more than $\lfloor n/2 \rfloor$ times in an array.

Pseudocode:

```
count=0
for num
```

```
    if count==0 candidate=num
    if num==candidate count++
    else count--
```

---

35. LC 171 – Excel Sheet Column Number

Description: Convert an Excel column title (like "AB") into its corresponding column number.

Pseudocode:

```
result = 0
for char
    result = result*26 + value
```

---

36. LC 190 – Reverse Bits

Description: Reverse the bits of a given 32-bit unsigned integer.

Pseudocode:

```
for 32 bits
    result = (result<<1) | (n&1)
    n >>=1
```

---

37. LC 191 – Number of 1 Bits

Description: Count the number of set bits (1s) in the binary representation of a number.

Pseudocode:

```
count = 0
while n
    n = n & (n-1)
    count++
```

---

38. LC 202 – Happy Number

Description: Determine whether a number is happy by repeatedly replacing it with the sum of squares of its digits.

Pseudocode:

```
use set
repeat sum of squares
```

---

39. LC 203 – Remove Linked List Elements

Description: Remove all nodes from a linked list that have a specific value.

Pseudocode:

```
dummy.next = head
skip nodes with value
```

---

40. LC 205 – Isomorphic Strings

Description: Check whether two strings are isomorphic, meaning characters map one-to-one.

Pseudocode:

```
map chars both ways
```

---

41. LC 217 – Contains Duplicate

Description: Check whether any value appears at least twice in an array.

Pseudocode:

```
use set, if exists return true
```

---

42. LC 219 – Contains Duplicate II

Description: Check if two identical numbers exist within a given index distance k.

Pseudocode:

```
map value -> index
check distance <=k
```

---

43. LC 226 – Invert Binary Tree

Description: Invert a binary tree by swapping left and right children at every node.

Pseudocode:

```
swap left and right recursively
```

---

44. LC 231 – Power of Two

Description: Check whether a given integer is a power of two.

Pseudocode:

```
if n>0 and n&(n-1)==0
```

---

45. LC 242 – Valid Anagram

Description: Determine whether two strings are anagrams of each other.

Pseudocode:

```
count chars
compare
```

---

46. LC 258 – Add Digits

Description: Repeatedly add digits of a number until a single-digit result is obtained.

Pseudocode:

```
while n>=10
    n = sum of digits
```

---

47. LC 268 – Missing Number

Description: Find the missing number from an array containing numbers from 0 to n.

Pseudocode:

```
sum = n*(n+1)/2
subtract array sum
```

---

48. LC 283 – Move Zeroes

Description: Move all zero elements to the end of the array while maintaining order of non-zero elements.

Pseudocode:

```
index=0
move non-zero forward
```

---

49. LC 344 – Reverse String

Description: Reverse a character array in-place.

Pseudocode:

```
swap start and end
```

---

50. LC 383 – Ransom Note

Description: Check if a ransom note can be constructed using letters from a magazine string.

Pseudocode:

```
count magazine chars
reduce for ransom
```

Connect & Share:

Tag us on your success stories:

- **Instagram**: @code.abhii07
- **YouTube**: SYNTAX ERROR